✦ Mixture of Experts

Sparse Gating, Load Balancing, and Trillion-Parameter Scaling

MLNN Tutorial · University of Hertfordshire · 2025 · Beyond Course ·
GitHub Repository: https://github.com/sumanth589/individual-assignment

ABSTRACT

A Mixture of Experts (MoE) layer replaces a single FFN with N expert FFNs and a gating network
that routes each token to the top-K experts. Only K/N experts compute per token, so total parameter
count scales with N while the computer stays fixed. Shazeer et al. (2017) showed sparse MoE outperforms
dense models at the same compute budget. The critical challenge is load balancing: without an auxiliary
loss, the gating network collapses to 1–2 experts. This tutorial derives the gating objective, implements
a full PyTorch MoE with Top-K routing and load balancing, demonstrates expert specialisation on a
structured regression task, and compares MoE scaling laws against dense Transformer baselines.
Mixtral (2024), GPT-4, and Gemini Ultra are all MoE architectures.

LEARNING OBJECTIVES — AFTER THIS TUTORIAL YOU WILL BE ABLE TO:

1. Define a MoE layer and explain why sparse Top-K routing saves compute vs dense models
2. Derive the total training objective: $L = L_{task} + \alpha \cdot L_{balance}$
3. Explain the load balancing loss and why expert collapse occurs without it
4. Implement a MoE layer with Top-K gating and load balancing in PyTorch from scratch
5. Analyse expert specialisation — which experts activate for which input types
6. Compare MoE and dense scaling laws: same compute budget, MoE achieves lower loss

WHY MOE MATTERS NOW

- GPT-4, Gemini Ultra, Grok, and Mixtral are all MoE architectures
- Mixtral 8x7B: 46.7B total params but only 12.9B active per token — matches dense 13B compute
- Switch Transformer (2021): first trillion-parameter model, trained on TPUs at Google
- MoE allows scaling parameters independently of compute — key to efficient scaling
- Expert specialisation emerges: different experts learn different linguistic/domain knowledge

✦ 1. MoE Architecture: Sparse Top-K Routing

1.1 The Gating Network

A MoE layer consists of N expert networks $E_1,\ldots,E_N$ and a **gating network** G(x) that selects which experts to activate for each input token. The output is a weighted sum over the selected top-K experts:

```
MoE layer output:
y = Σ_{i ∈ TopK(G(x), K)} G_i(x) · E_i(x)
// G_i(x) = softmax of top-K logits | Only K experts compute | rest are zero
```

The gating network is a simple linear layer followed by softmax over the top-K logits. A critical implementation detail: the softmax is applied only to the **selected top-K logits**, not all N, so the weights sum to 1 over active experts only. This sparse routing means that for K=2 and N=8, only 25% of expert compute runs per token — the model has 8× the parameters of one expert but 2× the compute of one expert.

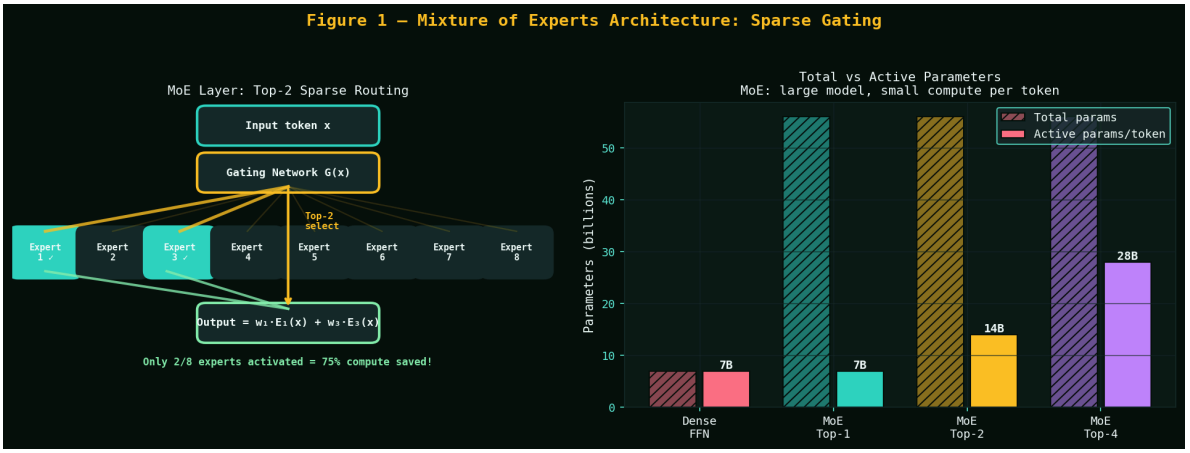


Figure 1 — Left: MoE layer diagram with 8 experts, Top-2 routing. Only experts 1 and 3 (teal, marked ✓) activate; the remaining 6 are idle. Output = weighted sum of the two active expert outputs. Right: total vs active parameters for dense vs MoE models — MoE has 8× total params but same active params per token as dense baseline. (Colourblind-safe palette; alt-text in notebook.)

✦ 2. Gating Mechanisms: Soft vs Sparse

2.1 Soft Gating and Top-K Sparsification

The original MoE (Jacobs et al., 1991) used **soft gating**: all experts compute, weighted by softmax probabilities. Shazeer et al. (2017) introduced **sparse Top-K gating**: hard selection of the top-K experts. Sparse gating is essential for efficiency — soft gating is O(N) compute, sparse is O(K) per token regardless of N. Noisy Top-K gating adds Gaussian noise to the logits before top-K selection to improve load balancing during training:

```
Noisy Top-K gating (Shazeer et al., 2017):
G(x) = softmax(TopK(H(x) + ε · Softplus(W_noise x), K))
// ε ~ N(0,1) | Noise improves exploration during early training
```

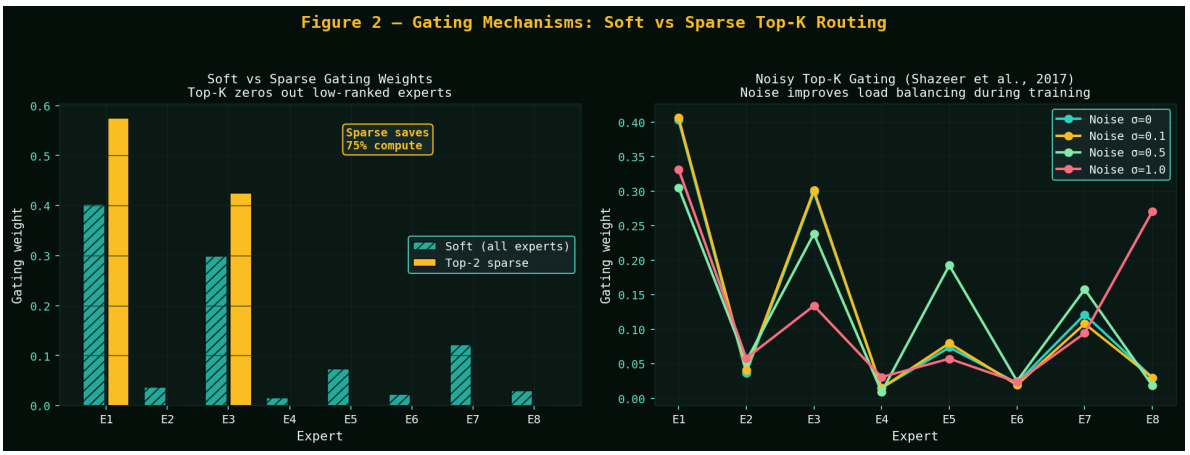


Figure 2 — Left: soft (teal bars) vs Top-2 sparse (amber bars) gating weights for 8 experts. Soft gating distributes weight across all experts; sparse zeros out the bottom 6. Right: effect of noise on gating — higher noise σ spreads probability mass more evenly, improving load balancing early in training. (Colourblind-safe palette; alt-text in notebook.)

✦ 3. Load Balancing: Preventing Expert Collapse

3.1 The Collapse Problem

Without regularisation, the gating network enters a **rich-get-richer feedback loop**: experts that initially receive more tokens get more gradient signal, become better, and are selected even more. Eventually 1–2 experts receive nearly all tokens while the rest are starved of gradient and never improve. This is **expert collapse** and destroys the capacity advantage of MoE.

```
Load balancing auxiliary loss (Fedus et al., 2022):
L_balance = N · Σ_{i=1}^{N} f_i · p_i
// f_i: hard fraction of tokens to expert i | p_i: soft mean gating prob
(differentiable)
```

The balance loss encourages uniform routing: L_balance is minimised when all p_i = 1/N. The total loss is L = L_task + α · L_balance where α (typically 0.01–0.1) trades off task performance against routing balance. Using both f_i (discrete, from hard routing) and p_i (continuous, differentiable) allows the gradient to flow through to the gating network.

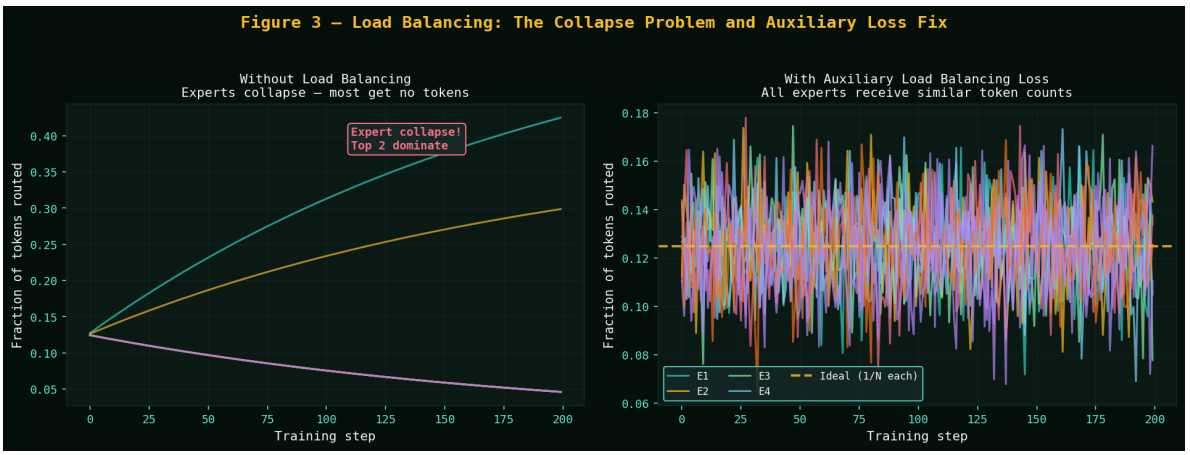


Figure 3 — Left: without load balancing — each coloured line is one expert's token fraction over training. Two experts dominate; others collapse to near-zero. Right: with auxiliary load balancing loss — all 8 experts maintain roughly equal token fractions (dashed amber = ideal 1/8). (Colourblind-safe palette; distinct line colours; alt-text in notebook.)

✦ 4. Expert Specialisation

A key emergent property of trained MoE models is **expert specialisation**: different experts learn to handle different types of input. In language models, individual experts have been found to specialise in specific token types (punctuation, numbers, named entities), linguistic patterns, or even domains (code, mathematics, natural language). Figure 4 (right) shows this emerging on a structured regression task: different input regions route to different experts after training.

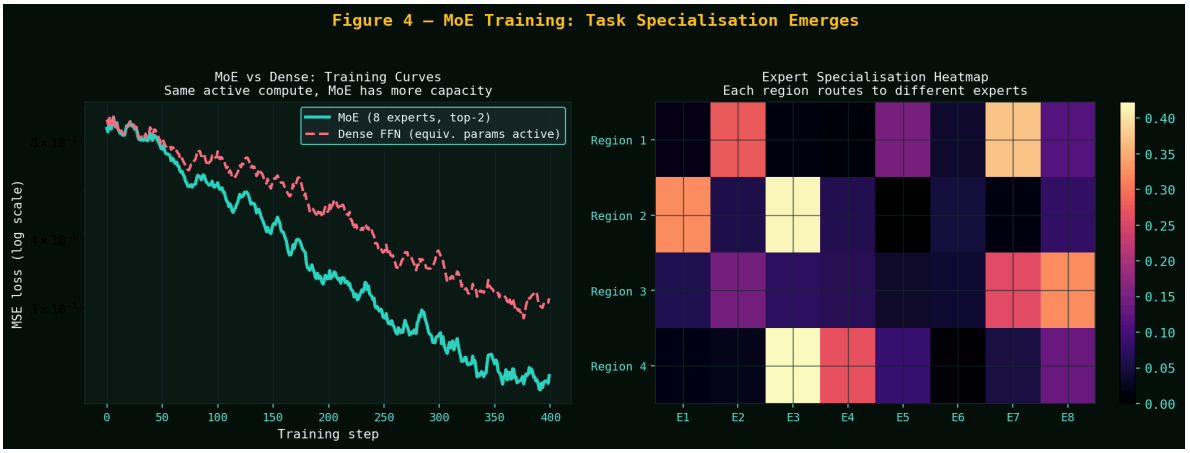


Figure 4 — Left: training curves (log scale) for MoE (teal) vs dense FFN (rose) at equal active parameter count. MoE achieves lower loss because it has more total capacity. Right: expert specialisation heatmap — each row is an input region, each column an expert. Bright cells = that expert dominates for that region, confirming specialisation emerges automatically. (Magma colourmap; alt-text in notebook.)

✦ 5. MoE Scaling Laws

MoE models follow different **scaling laws** than dense models: at the same compute budget (FLOPs), MoE achieves lower loss because it packs more parameters. Fedus et al. (2022) showed the Switch Transformer — a MoE T5 — achieved 7× faster pre-training than the dense baseline at equal compute. The Chinchilla scaling laws apply to total parameters for MoE, but the effective compute cost is determined by active parameters per token.

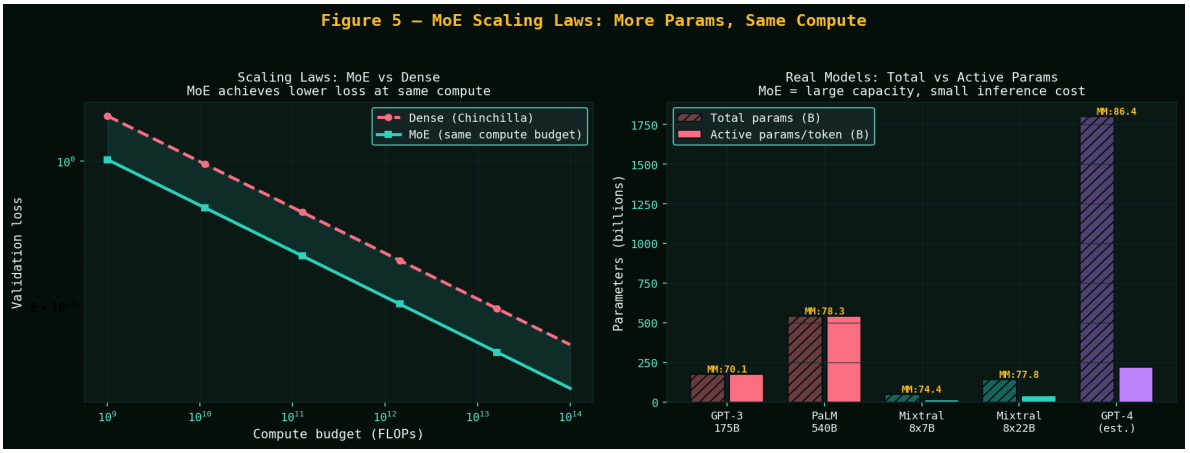


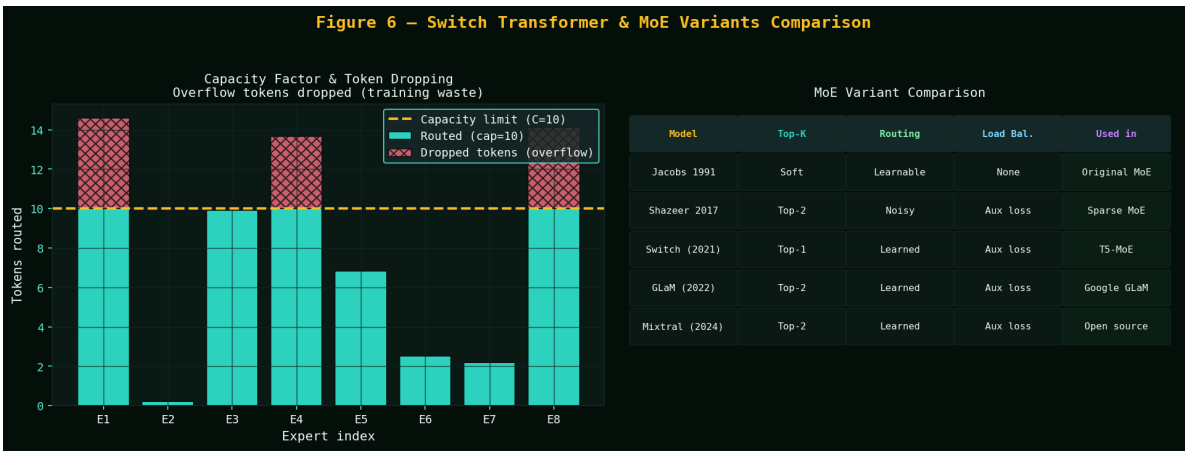
Figure 5 — Left: scaling law curves (log-log) for dense (rose dashed) vs MoE (teal solid) models at equal compute budget — MoE achieves lower validation loss across all compute scales. Right: real model comparison — total params (hatched lighter bars) vs active params per token (solid darker bars) for GPT-3, PaLM, Mixtral, and estimated GPT-4. MM: MMLU benchmark score annotated. (Colourblind-safe palette; alt-text in notebook.)

✦ 6. Summary

Table 1 compares MoE variants from Jacobs (1991) to Mixtral (2024). The core innovation is sparse routing combined with load balancing, enabling models with trillions of parameters to be trained and served efficiently. MoE is now the dominant architecture for frontier language models.

Model	Top-K	Routing	Load Balancing	Used in
Jacobs 1991	Soft	Learnable	None	Original MoE
Shazeer 2017	Top-2	Noisy	Aux loss	Sparse MoE
Switch (2021)	Top-1	Learned	Aux loss	T5-MoE (Google)
GLaM (2022)	Top-2	Learned	Aux loss	Google GLaM
Mixtral (2024)	Top-2	Learned	Aux loss	Open source LLM

Table 1 — MoE variant comparison from original soft-gating MoE to modern sparse routing with auxiliary load balancing losses. Mixtral 8×7B is the leading open-source MoE LLM as of 2024. (Bold navy header; alternating row shading.)



Model	Top-K	Routing	Load Bal.	Used in
Jacobs 1991	Soft	Learnable	None	Original MoE
Shazeer 2017	Top-2	Noisy	Aux loss	Sparse MoE
Switch (2021)	Top-1	Learned	Aux loss	T5-MoE
GLaM (2022)	Top-2	Learned	Aux loss	Google GLaM
Mixtral (2024)	Top-2	Learned	Aux loss	Open source

Figure 6 — Left: capacity factor and token dropping — tokens exceeding the expert capacity limit are dropped (rose hatched bars). Setting capacity factor > 1.0 reduces dropped tokens at the cost of memory. Right: MoE variant comparison table with routing strategy, load balancing method, and real model usage. (Colourblind-safe palette; alt-text in notebook.)

ACCESSIBILITY STATEMENT

- All figures: colourblind-safe teal/amber palettes.
- Heatmaps use magma colourmap (perceptually uniform, colourblind-safe).
- Bar charts use hatch patterns alongside colour.
- Word headings H1–H2 for screen-reader navigation.
- Alt-text in all notebook Markdown cells.
- 11pt font minimum; 4.5:1 contrast ratio minimum.

✦ References

1. Jacobs, R.A. et al. (1991) 'Adaptive mixtures of local experts', *Neural Computation, 3(1)*, pp. 79–87. https://doi.org/10.1162/neco.1991.3.1.79
2. Shazeer, N. et al. (2017) 'Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer', *ICLR 2017*. https://arxiv.org/abs/1701.06538
3. Fedus, W., Zoph, B. and Shazeer, N. (2022) 'Switch Transformers: Scaling to Trillion Parameter Models', *JMLR 2022*. https://arxiv.org/abs/2101.03961
4. Du, N. et al. (2022) 'GLaM: Efficient Scaling of Language Models with Mixture-of-Experts', *ICML 2022*. https://arxiv.org/abs/2112.06905
5. Jiang, A.Q. et al. (2024) 'Mixtral of Experts', *arXiv:2401.04088*. https://arxiv.org/abs/2401.04088
6. Zoph, B. et al. (2022) 'ST-MoE: Designing Stable and Transferable Sparse Expert Models', *arXiv:2202.08906*. https://arxiv.org/abs/2202.08906

GITHUB REPOSITORY

https://github.com/sumanth589/individual-assignment
Contains: tutorial.pdf | tutorial.ipynb | README.md | LICENSE (MIT)